DATA305 / DATA475 · Machine Learning Methods

CNN Architectures

ResNet · EfficientNet · Modern Design Patterns

From deeper networks to practical pretrained backbones

Lecture 8

Week 8 · Google Colab ready

Binh Nguyen · School of Mathematics and Statistics · Victoria University of Wellington

Today's Agenda & Learning Objectives

Lecture 8

Five sections · CIFAR-10 demos · pretrained Keras Applications

01

The Degradation Problem

Why naive depth can hurt training.

02

Skip Connections

ResNet idea, bottleneck blocks, gradient flow.

03

Depthwise Separable Conv

MobileNet's efficiency trick.

04

Compound Scaling

EfficientNet B0–B7: depth × width × res.

05

Keras Applications

Pretrained backbones; feature extraction.

Learning Objectives

- ✓ Why skip connections help deep nets train.
- ✓ Compare ResNet, MobileNet, EfficientNet.
- ✓ Use keras.applications as a feature extractor.

Datasets: keras.datasets.cifar10 · Models: keras.applications · Géron Ch. 14, pp. 495–520

Lecture 7 → Lecture 8

Bridge

In L7 we built a 3-block CNN reaching ~82% validation accuracy on CIFAR-10. Today we ask: when we keep stacking layers, does the network keep getting better?

Lecture 7 — Covered

- ✓ Local connectivity, weight sharing, equivariance
- ✓ Conv arithmetic: kernel · stride · padding · dilation
- ✓ Pooling: max · average · global average
- ✓ Feature maps and growing receptive fields
- ✓ 3-block CNN on CIFAR-10 → ~82% val acc

Lecture 8 — Today

- → The degradation problem in plain deep nets
- → Skip connections: residual learning intuition
- → Bottleneck blocks; ResNet-34 / 50 / 101 / 152
- → Depthwise separable convs (MobileNet)
- → Compound scaling: EfficientNet B0–B7
- → keras.applications: pretrained weights

Today's question: what happens when we scale CNN depth?

Running example: CIFAR-10 · pretrained weights download automatically (~20–100 MB)

01

The Degradation Problem

Plain deep nets · 56 vs 20 layers · CIFAR-10 demo · setup for skip connections

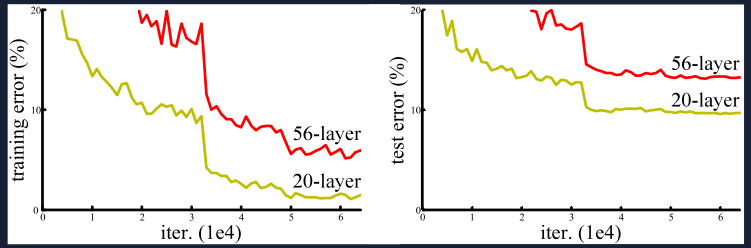
The Degradation Problem

The root puzzle

Adding layers should never hurt — a deeper plain net can always mimic a shallower one by setting the extra layers to identity mappings.

Experiments showed the opposite:

- A 56-layer plain net has higher training error than a 20-layer plain net.
- This is not overfitting — the training error itself is worse.
- Plain deep nets are simply hard to optimize: it is not only about gradient flow — it is also hard for them to learn identity mappings.



Plain-56 has higher training error than Plain-20 (CIFAR-10)

He et al. (2015)

Deeper plain nets are harder to optimize, not just harder to regularize. Residual connections make the identity mapping easy to represent.

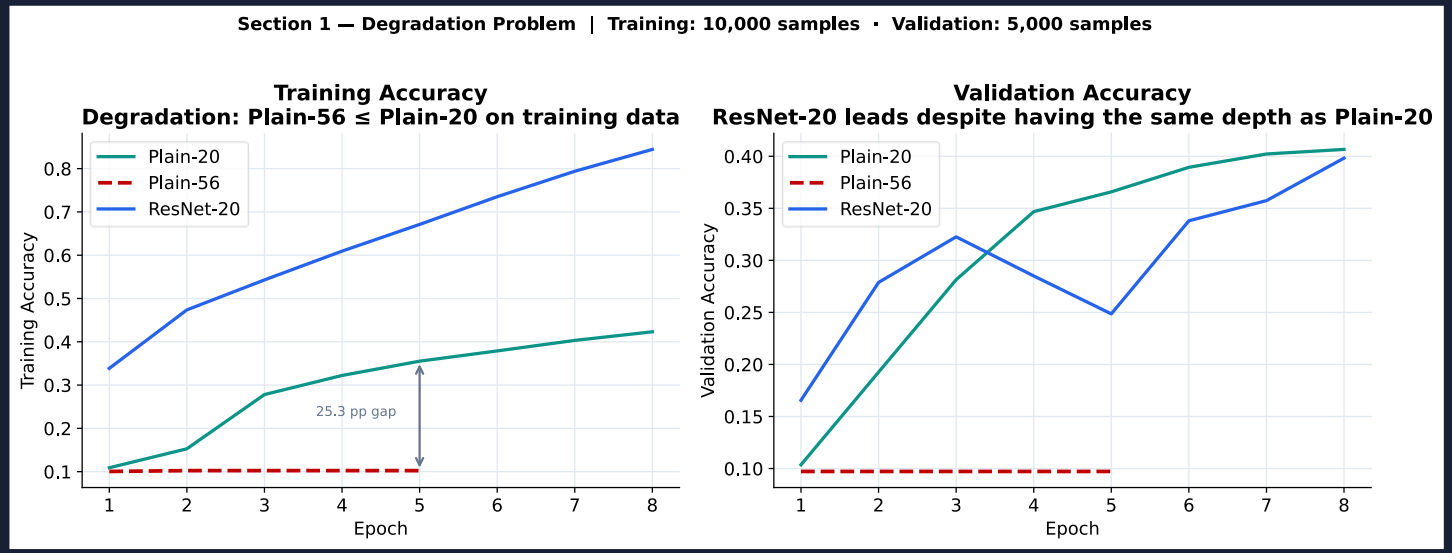
ImageNet ILSVRC 2015

ResNet won with 3.57% top-5 error.

152-layer ResNet was 8× deeper than VGG.

Degradation Demo — CIFAR-10 results

Plain-20 train **0.42** | Plain-56 train **0.10** | ResNet-20 train **0.84** → *degradation confirmed*



Plain CNNs intentionally omit BN to make degradation visible in a short run. This is an instructional demonstration, not a controlled architecture benchmark.

Residual Learning — Intuition & Math

Residual learning formulation

Plain block learns: $H(x)$ directly

Residual block learns: $F(x) = H(x) - x$

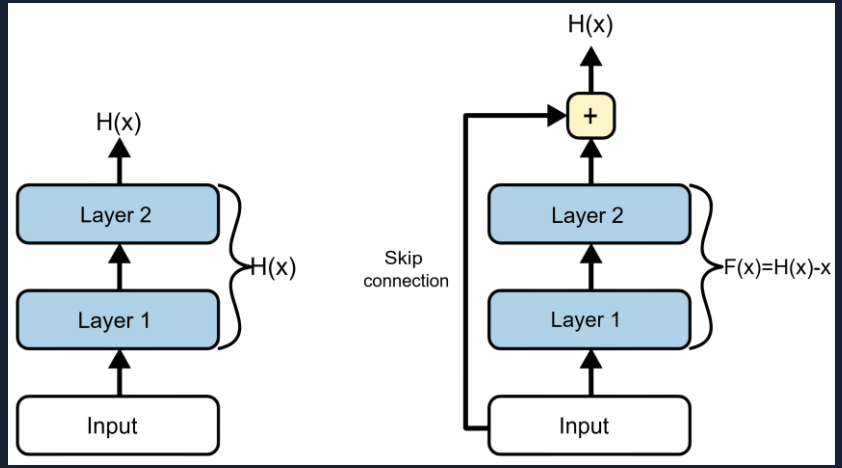
Output: $H(x) = F(x) + x$

Near initialization, F ≈ 0 → the block ≈ identity, so gradients flow through the skip path.

Key insight

Gradient: $\partial L/\partial x = \partial L/\partial H \cdot (1 + \partial F/\partial x)$

The +1 term is a direct gradient path through the identity shortcut.



Left: plain block H(x). Right: residual unit F(x) = H(x) − x + skip.

Identity shortcut: no extra parameters

Projection shortcut (1×1 conv) is used only when shape changes — that path does add parameters.

CNN Architecture Timeline

AlexNet (2012) → VGG (2014) → Inception (2014) → ResNet (2015) → MobileNet (2017) → EfficientNet (2019). Each one introduced a lasting design idea.

Architecture	Year	Layers	Params	Key design idea
AlexNet	2012	8	60 M	ReLU · Dropout · GPU training
VGGNet	2014	16–19	138 M	Small 3×3 filters stacked deep
GoogLeNet	2014	22	6 M	1×1 bottleneck · multi-scale Inception
ResNet-50	2015	50	25 M	Skip connections — this lecture
MobileNetV2	2018	53	3.4 M	Depthwise separable conv — this lecture
EfficientNet-B7	2019	66	66 M	Compound scaling — this lecture

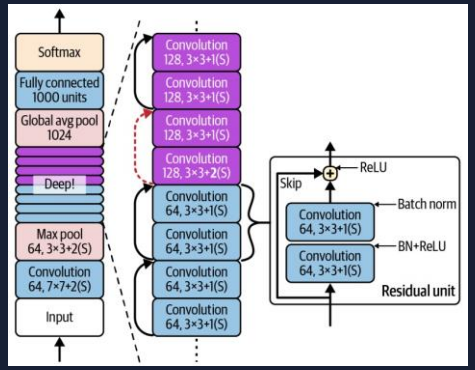
ImageNet top-5 error trajectory: AlexNet 16.4% → VGG 7.3% → Inception 6.7% → ResNet 3.57% → EfficientNet-B7 ~2.9%

02

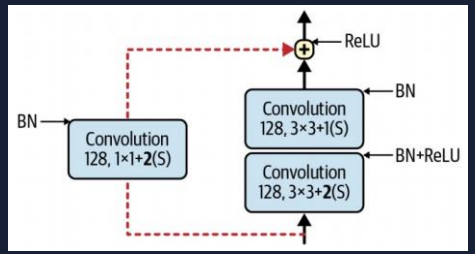
Skip Connections

Residual unit · bottleneck blocks · ResNet family · gradient flow

Residual Unit — Anatomy



ResNet stacks residual units with skip connections



1×1 conv shortcut when stride or depth changes

Anatomy of a residual unit

1 **Conv 3×3 → BN → ReLU**

3×3 kernel preserves spatial size (same padding).

2 **Conv 3×3 → BN (no ReLU yet)**

BatchNorm after the last conv, before adding the skip.

3 **Add the skip: H(x) = F(x) + x**

Element-wise addition — x must match shape.

4 **ReLU after the addition**

Activation comes last in the post-activation pattern.

Shape must match before addition. No pooling inside the unit; downsampling is done with stride-2 conv at stage transitions.

Bottleneck Block — 1×1 → 3×3 → 1×1

Standard residual unit (left): two 3×3 convolutions. Bottleneck unit (right): 1×1 → 3×3 → 1×1.

How it saves compute

1×1 squeeze reduces channels → the expensive 3×3 conv operates on far fewer feature maps.

1×1 expand restores the channel count for the next block.

Parameter cost — notebook §2.2

256 filters: standard **1.18 M** vs bottleneck **69.6 K**

→ ≈ **17×** fewer parameters per block at this width.

Similar savings occur at other channel widths; the savings let ResNet-50/101/152 grow deeper without exploding cost.

Géron Ch. 14, p. 508 — bottleneck design used by ResNet-50, -101, -152

ResNet Family — 34 / 50 / 101 / 152

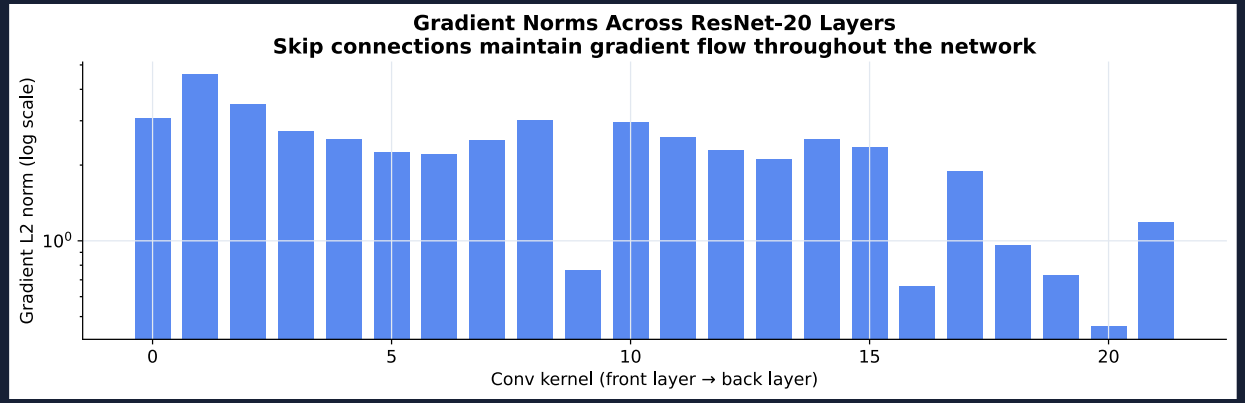
Same architectural idea, different depths. ResNet-34 keeps the standard block; ResNet-50 and deeper switch to bottleneck blocks.

Model	Layers	Residual units	Block type	Params (notebook)
ResNet-34	34	16	Standard (3×3 → 3×3)	~21.8 M
ResNet-50	50	16	Bottleneck (1×1 → 3×3 → 1×1)	~25.6 M
ResNet-101	101	33	Bottleneck (1×1 → 3×3 → 1×1)	~44.5 M
ResNet-152	152	50	Bottleneck (1×1 → 3×3 → 1×1)	~60.2 M

CIFAR-style ResNets often use 16→32→64 channels. ImageNet ResNets use larger stages such as 64→128→256→512; bottleneck blocks expand the output depth.

Practical use: keras.applications.ResNet50(weights="imagenet") — see Section 05

Gradient Flow — ResNet-20 (notebook §2.1)



L2 norm of gradients in each Conv2D layer after one backward pass.

Measured on ResNet-20

Min norm

0.456

Max norm

4.57

Max / Min ratio

10.0×

Compared with the vanishing-gradient examples from earlier lectures, this ResNet-style model maintains much healthier gradient norms across depth.

Skip connections add an identity path that lets gradients reach early layers without compounding small derivatives.

ResNet-20 trained on the CIFAR-10 subset from §1

Implementing ResNet — one method

In Keras the residual unit is a custom Layer. The key line is the addition: return relu(Z + skip_Z).

```
class ResidualUnit(tf.keras.layers.Layer):
    def __init__(self, filters, strides=1, **kwargs):
        super().__init__(**kwargs)
        self.main = [
            DefaultConv2D(filters, strides=strides),
            tf.keras.layers.BatchNormalization(),
            tf.keras.layers.Activation("relu"),
            DefaultConv2D(filters),
            tf.keras.layers.BatchNormalization(),
        ]
        # 1x1 projection on the skip path only when shape changes
        self.skip = []
        if strides > 1:
            self.skip = [
                DefaultConv2D(filters, kernel_size=1, strides=strides),
                tf.keras.layers.BatchNormalization(),
            ]

    def call(self, x):
        Z = x
        for layer in self.main: Z = layer(Z)
        skip_Z = x
        for layer in self.skip: skip_Z = layer(skip_Z)
        return tf.keras.activations.relu(Z + skip_Z)   # the residual idea
```

`Z + skip_Z`

The residual idea is one line of code: add the input back to the output of the conv block.

`self.skip`

Empty when shapes match — a 1×1 conv is added only when stride or depth changes.

In practice

Implement once to understand; use keras.applications.ResNet50(weights="imagenet") (§5).

03

Depthwise Separable Conv

Factorizing 3×3 conv · MobileNet V1 / V2 · on-device inference

Depthwise Separable Conv — Motivation

Mobile and edge deployment needs accurate CNNs in milliseconds. Standard convolutions are expensive — can we factor them?

Standard convolution cost

One Conv2D with kernel $k \times k$, C_in inputs and C_out outputs:

$$\text{Cost} = k \cdot k \cdot C_in \cdot C_out \text{ parameters}$$

Example 3×3, 256→256 → **589 824** params per layer.

Every output channel uses a 3-D kernel across all input channels.

Depthwise separable trick

Factor into two cheap steps:

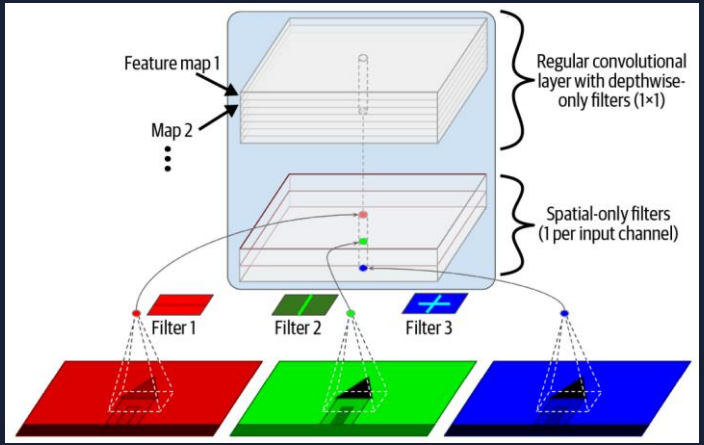
1. **Depthwise:** one 3×3 filter per input channel
   $\text{Cost} = 3 \cdot 3 \cdot C_in$
2. **Pointwise:** 1×1 conv to mix channels
   $\text{Cost} = C_in \cdot C_out$

Total ≈ **8 – 9× fewer** operations (3×3 example).

A useful factorisation that often preserves accuracy while greatly reducing compute.

Depthwise Separable Conv — Diagram

Factor a 3×3 × C_in × C_out conv into two cheaper steps.



Standard conv jointly mixes spatial and channel information. Depthwise separable conv factors this into depthwise spatial filtering + 1×1 channel mixing.

Parameter comparison (Cin = Cout = 256): **Standard 589 824** | **Depthwise separable 67 840** → *≈ 8.7× fewer*

Fig 14-20 (Géron) · Same factorisation underpins Xception and MobileNet

MobileNet — Efficient On-Device CNNs

MobileNet V1 (Howard 2017)

Replaces most standard convolutions with depthwise separable blocks.

About 28 depthwise separable layers.

Width multiplier $\alpha \in \{1.0, 0.75, 0.5, 0.25\}$ trades accuracy for speed.

Resolution multiplier ρ: smaller inputs for lower latency.

MobileNet V2 (Sandler 2018)

Inverted residual block: expand channels → depthwise → project down.

Linear bottleneck — no ReLU after the final projection (avoids information loss in low-dim space).

Skip connection on the bottleneck (residual), not on the expansion.

ImageNet top-1 (typical): MobileNetV2 71.3% · ResNet-50 74.9% · EfficientNetB0 77.1% → MobileNet is much smaller for a few percent of accuracy.

```
m = keras.applications.MobileNetV2(weights="imagenet", include_top=False, input_shape=(224, 224, 3))
```

Howard 2017 · Sandler 2018 · Géron Ch. 14, p. 511

04

EfficientNet Compound Scaling

Scaling depth · width · resolution jointly · EfficientNet-B0 to B7

EfficientNet — Three Scaling Dimensions

ResNet trains deep networks reliably. When we have more compute, which dimension matters most?

Depth

More layers → larger receptive field, more abstract features.

Pro: hierarchical features.

Con: very deep plain nets degrade — fixed by skip connections.

Width

More filters per layer → richer features at each resolution.

Pro: fine-grained discrimination.

Con: wide-shallow nets saturate; expensive in memory.

Resolution

Larger input → finer spatial detail.

Pro: better for small objects.

Con: compute grows with the square of resolution.

Scaling one dimension alone gives diminishing returns. EfficientNet's idea: scale all three jointly using a principled compound coefficient φ.

EfficientNet — Compound Scaling Rule

Earlier work scaled depth OR width OR resolution. EfficientNet scales all three jointly with one coefficient.

Compound scaling

$$d = \alpha^{\varphi}$$

depth

$$w = \beta^{\varphi}$$

width

$$r = \gamma^{\varphi}$$

resolution

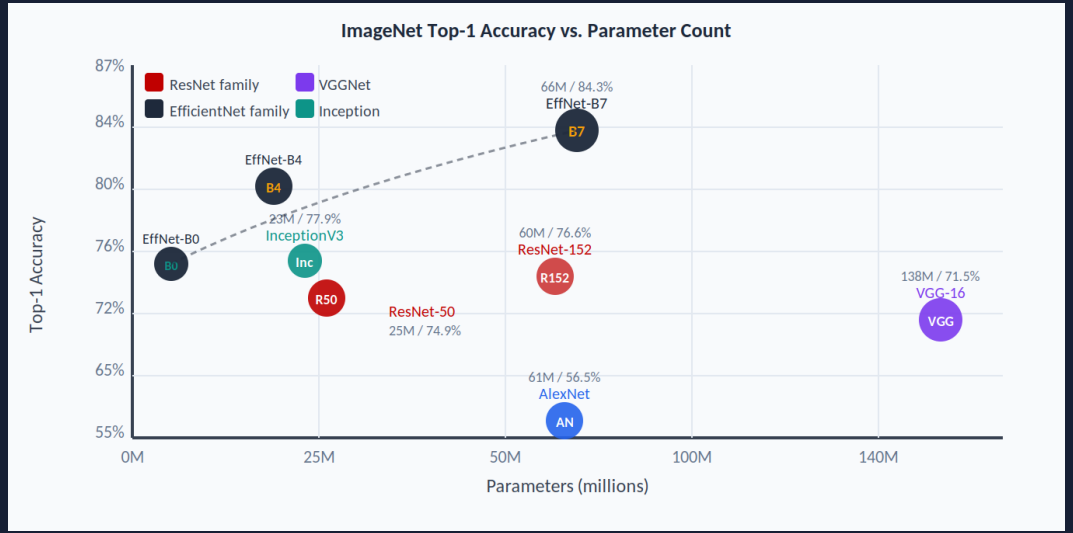
Constraint (NAS-tuned)

$$\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2 \quad \text{with} \quad \alpha = 1.2,\ \beta = 1.1,\ \gamma = 1.1$$

Increasing φ by 1 doubles the FLOPs. **B0 = baseline** ($\phi = 0$). **B1 – B7** step ϕ up.

Tan & Le 'EfficientNet: Rethinking Model Scaling' ICML 2019

EfficientNet Family — B0 to B7



Accuracy vs. parameter count: EfficientNet sets a strong frontier.

Model	Params	Input	Top-1
B0	5.3 M	224	77.1%
B1	7.8 M	240	79.1%
B2	9.2 M	260	80.1%
B3	12 M	300	81.6%
B4	19 M	380	82.9%
B5	30 M	456	83.6%
B6	43 M	528	84.0%
B7	66 M	600	84.3%

B0 = practical baseline · B7 = high-accuracy

EfficientNet-B7: 84.3% top-1 with 66 M params vs. AlexNet ~56.5% top-1 with ~60 M params → +27.8 pp at similar scale.

EfficientNet checkpoints are available via keras.applications; weights download automatically on first use.

05

Keras Applications

Pretrained backbones · feature extraction · fine-tuning preview (full in L9)

Why Pretrained Models?

Training a deep CNN from scratch on small data is hard. Pretrained backbones give us a head start.

Training from scratch

- ✗ Needs a very large labeled dataset (≫ 50 K images).
- ✗ Can take days on multi-GPU hardware.
- ✗ Heavy hyper-parameter tuning (LR schedule, augmentation, regularization).
- ✗ Risk of overfitting on small task data.
- ✗ Reinventing features that ImageNet has already learned.

What keras.applications gives you

- ✓ Backbones already trained on ImageNet (≈ 1.3 M images, 1 000 classes).
- ✓ Generic visual features (edges, textures, shapes, object parts).
- ✓ Models load in seconds; weights are tens to hundreds of MB.
- ✓ Same Keras API as any other model (Layer / Model).
- ✓ Drop-in starting point for transfer learning.

keras.applications — Model Zoo

A library of well-known architectures with pretrained ImageNet weights — all behind one consistent API.

ResNet50
Skip connections

ResNet101
Deeper variant

ResNet152
Deeper still

MobileNetV2
Depthwise separable

EfficientNetB0
Compound scaling

EfficientNetB7
Largest variant

InceptionV3
Multi-scale (appendix)

DenseNet121
Dense connections

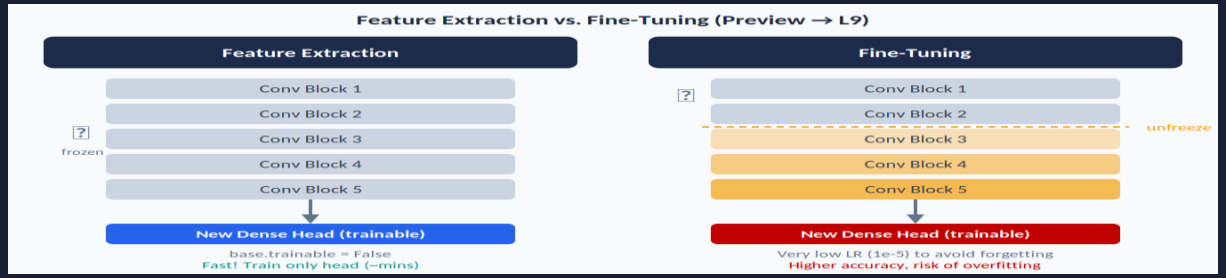
Xception
All depthwise sep.

```
base = keras.applications.MODEL(weights="imagenet", include_top=False, input_shape=(H, W, 3))
```

One API, many backbones · swap MODEL → ResNet50 / EfficientNetB0 / MobileNetV2 / …

Two Ways to Use a Pretrained Backbone

Today we focus on feature extraction. Fine-tuning is a Lecture 9 topic — we only preview the idea.



Feature Extraction (today)

Freeze the backbone. Train only a small new head on your task labels.

Few trainable params → fast and works on small data.

Today: feature extraction only.

Fine-Tuning (preview — L9)

Unfreeze top backbone layers; continue training with very small LR.

Few extra accuracy points — but easier to overfit if mis-tuned.

Watch BatchNorm modes — details in L9.

Feature Extraction — minimal Keras code

Load a backbone, freeze it, attach a small classification head, train.

```
base = keras.applications.EfficientNetB0(
    weights="imagenet", include_top=False, input_shape=(96, 96, 3)
)
base.trainable = False                          # freeze the pretrained weights

inputs  = keras.Input(shape=(96, 96, 3))
x       = keras.applications.efficientnet.preprocess_input(inputs)
x       = base(x, training=False)               # backbone in inference mode
x       = layers.GlobalAveragePooling2D()(x)
outputs = layers.Dense(10, activation="softmax")(x)

model = keras.Model(inputs, outputs)
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
```

`include_top=False`

Drops the 1000-class ImageNet head — we add our own.

`base.trainable = False`

Freezes backbone weights — only the head is updated.

Input size = 96×96

Notebook uses 96×96 to match its CIFAR-10 setup. EfficientNetB0's canonical ImageNet input is 224×224.

Notebook adds Dropout(0.3) + Dense(256, ReLU, L2) before the final 10-class head.

Notebook §3.2 · EfficientNetB0 uses float pixels in [0,255]; preprocessing is built into the model. For other backbones, use the matching preprocess_input.

Notebook Result — pretrained vs. from scratch

Section 05

Same dataset, different model sizes and training budgets. Frozen EfficientNetB0 backbone + tiny head, trained for a few epochs.

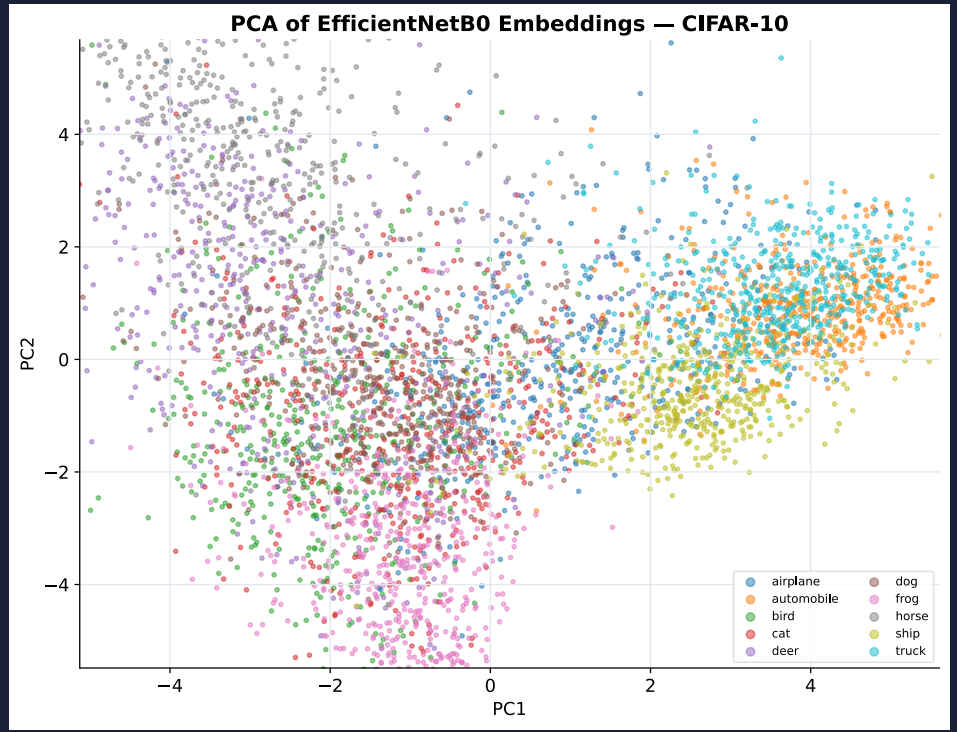
ResNet-20 (from scratch)	EfficientNetB0 (frozen)	Improvement
0.398	**0.877**	**+47.9 pp**
val accuracy after 8 epochs	*best val accuracy*	*absolute, illustrative not benchmark*

Why such a gap? EfficientNetB0 has already seen 1.3 M ImageNet images and learned generic visual features. Our small CIFAR-10 budget is far from enough to learn those from scratch.

Educational comparison — different model sizes and training budgets.

PCA of EfficientNetB0 embeddings

The penultimate layer is a 1280-dim vector per image. We project to 2-D with PCA and color by class.



Embedding shape

(1280,) — one vector per image. Same code regardless of which keras.applications model we use.

What the plot shows

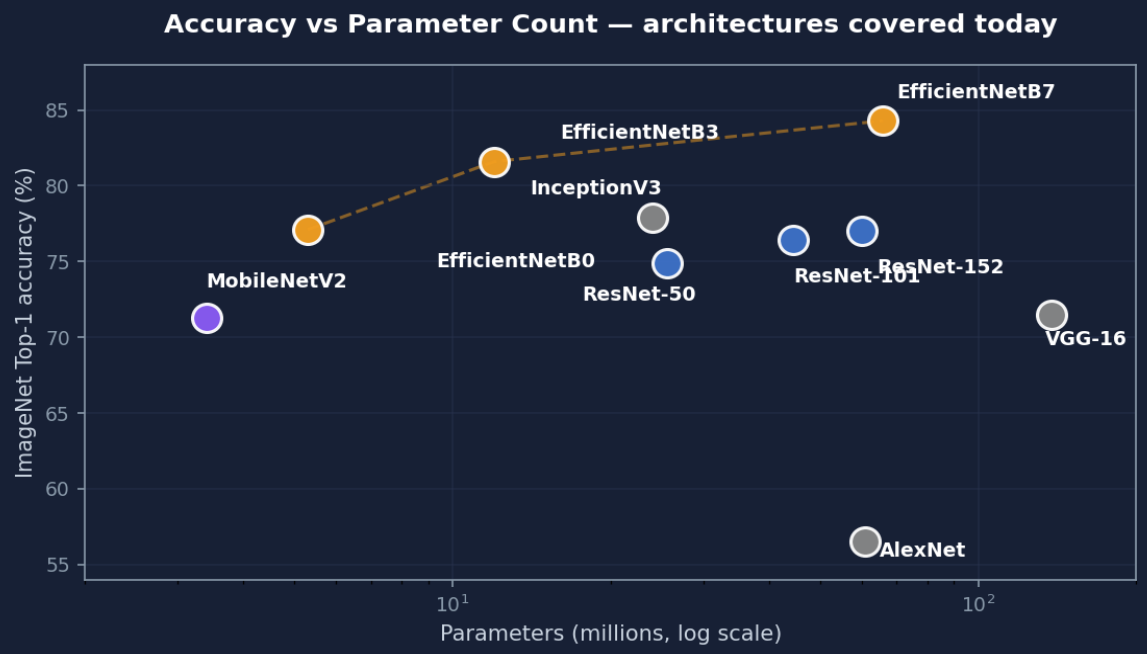
Some classes form visible structure even before fine-tuning, although overlap remains.

Code

```
emb_model = keras.Model(
    base.input,
    layers.GlobalAveragePooling2D()(base.output)
)
embeds = emb_model.predict(pca_ds)
```

Accuracy vs. Parameters — the big picture

The architectures we covered today on the same chart. Smaller and to the upper left is better.



ResNet

Reliable accuracy, larger params. Good default choice.

MobileNet

Tiny model, fast inference, modest accuracy hit.

EfficientNet

Sets a strong frontier of accuracy per parameter.

EfficientNet sets a strong frontier — but the right backbone depends on the deployment target

Putting It Together — Colab walkthrough

Section 05

End-to-end recipe used in the notebook. Open the Colab and follow along.

1

Load CIFAR-10

tf.keras.datasets.cifar10 → split train / val / test.

2

Build feature-extractor model

EfficientNetB0(weights="imagenet", include_top=False) · freeze.

3

Train just the head

GlobalAveragePooling2D + Dense(10, softmax) · Adam · a few epochs.

4

Inspect & evaluate

Accuracy curves · PCA of embeddings · comparison table.

Quiz Answer

Correct answer: B · Degradation — deeper plain nets are harder to optimize. Fix: skip connections.

Why not A — Overfitting?

Plain-56 has higher training error too, not just a train-vs-val gap. That's the opposite of overfitting.

Why not C / D?

Depthwise separable conv (C) is an efficiency trick, not a depth fix. The labels are fine — the same pipeline trains Plain-20 to 0.40.

Why B · the fix

Plain-56 fails because deep plain stacks are hard to optimize — gradients are unstable and identity mappings are hard to learn.

Adding skip connections $H(x) = F(x) + x$ gives a direct gradient path and lets each block learn a small residual on top of the identity. This usually makes optimization much easier *(notebook ResNet-20 train acc 0.770).*

Slide 6 results · Slide 13 gradient norms · Slide 14 implementation

Summary · Lecture 8

Wrap-up

01	Degradation	Naive depth hurts training — Plain-56 lags Plain-20 even on the training set.
02	Skip connections	F(x) + x makes identity mappings easy and keeps gradients flowing through depth.
03	Depthwise separable	Factor 3×3 conv into depthwise + 1×1 — the basis of MobileNet's efficiency.
04	Compound scaling	Scale depth × width × resolution jointly — EfficientNet sets a strong frontier.
05	Pretrained backbones	keras.applications gives instant access to ImageNet-trained features.

Lecture 9 preview

Transfer learning and fine-tuning in depth. Later: object detection and segmentation.